

# Basket Duel : Math et Physique du Jeu

## 1. Rôle et Périmètre d'Intervention

En tant que responsable du moteur physique et de la gestion spatiale, mon rôle est de modéliser le comportement réaliste des éléments du jeu Basket Duel. Cette responsabilité se situe au cœur du Modèle et garantit le bon fonctionnement des mécaniques de tir, de score et de génération de niveau.

Mon périmètre d'action couvre la conception et l'implémentation de quatre classes principales : Ballon, Terrain, Panier, et Bonus.

## 2. Modélisation Mathématique de la Trajectoire

L'élément central du gameplay est le calcul de la trajectoire du ballon. Plutôt que d'utiliser des animations précalculées, le jeu simule une trajectoire parabolique en temps réel basée sur les lois de la physique classique.

Lorsqu'un joueur valide son tir, il fournit un angle  $\theta$  et une puissance (vitesse initiale  $v_0$ ).

- **Décomposition du vecteur vitesse :**

$$v(x) = v_0 \cdot \cos(\theta)$$

$$v(y) = v_0 \cdot \sin(\theta)$$

- **Équations horaires du mouvement :**

À chaque instant  $t$  (mise à jour de la boucle de jeu), les nouvelles coordonnées  $(x, y)$  du ballon sont calculées en appliquant la constante de gravité ( $g$ ) :

$$x(t) = x_0 + v(x) \cdot t$$

$$y(t) = y_0 + v_y \cdot t - \frac{1}{2} \cdot g \cdot t^2$$

Cette approche permet de générer une courbe lisse et réaliste qui sera ensuite transmise à la Vue pour l'affichage graphique.

### 3. Système de Détection des Collisions

Pour que le jeu soit interactif, le système doit détecter quand le ballon rencontre d'autres éléments. J'ai mis en place un système de détection basé sur des boîtes de collision (hitboxes) circulaires.

- **Collision Ballon vs Panier / Bonus :**

La méthode utilise le calcul de la distance euclidienne  $d$  entre le centre du ballon  $(x_1, y_1)$  et le centre de la cible  $(x_2, y_2)$  :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Si cette distance  $d$  est inférieure ou égale à la somme des rayons des deux objets, la collision est avérée et l'événement correspondant est déclenché (marquage de points ou application d'un effet).

- **Collision Ballon vs Sol (Limites du terrain) :**

Une simple vérification des coordonnées  $y$  du ballon par rapport à la hauteur maximale du composant Terrain permet de détecter si le tir a échoué (tombé au sol).

### 4. Génération Spatiale Procédurale

Pour garantir une forte rejouabilité et forcer les joueurs à adapter leur stratégie (interactions stratégiques), le terrain est régénéré à chaque nouveau tour.

La classe Terrain inclut une logique de positionnement aléatoire sécurisée :

- **Placement du Panier :** Génération de coordonnées aléatoires  $(x, y)$  restreintes à une zone spécifique (ex: la moitié droite de l'écran) pour garantir que le tir reste physiquement possible.
- **Placement des Bonus :** Apparition aléatoire dans l'espace aérien situé entre le joueur et le panier.
- **Sécurité :** L'algorithme intègre une vérification pour empêcher le chevauchement des entités nouvellement créées (ex: un bonus ne peut pas apparaître exactement sur le panier).

## 5. Classes à Coder (Implémentation Java)

Pour réaliser ces fonctionnalités, voici la structure des entités dont j'ai la charge :

- Ballon : Stocke ses coordonnées temporelles, sa vitesse, et exécute la méthode de calcul de trajectoire.
- Terrain : Définit les limites physiques (murs, sol, gravité) et orchestre le placement des objets.
- Panier : Entité cible statique possédant des coordonnées et un rayon de collision.
- Bonus : Entité interactive générée procéduralement.